Exam 2 Summary

Annabelle Adachi
Comp 123

Booleans and Conditionals

bool: Python type for True/False values

- ★ Boolean values are not strings! - True not “True”

Boolean expression: expression that evaluates to a boolean value

- ★ Comparison operators:
  - == equal
  - != not equal
  - > greater
  - < less
  - >= greater than or equal to
  - <= less than or equal to
- ★ Logical operators:
  - **And** - only true if all parts are true
  - **Or** - true if one or more parts are true
  - **Not** - only true if all parts are false
- ★ Logical opposites:
  - ==/!=
  - </>=
  - >/<=

Conditional/selection statements:

- ★ **If** statement (binary selection)
- ★ If w/o else: unary selection - if not true, flow of execution continues to next statement after body of if statement
- ★ Conditionals can be nested within another conditional
- ★ Chained conditionals: use **elif** for more than two options

Unit Testing

Assert: runs code if precondition is true, otherwise stops code and returns error

Defensive function: function designed to check if parameters are valid

Unit test: code that tests if a function works properly

- ★ Manual unit test: gets input from user to test code

- ★ Automated unit test: contains its own test inputs, produces clear pass or fail result without user input
- ★ Unit tests can have bugs! Important to make sure unit test is correct

Testable function: function that produces a result which can be checked by an assert statement by returning a result, storing it in a global variable, or modifying the state of an object
Postcondition: what the function completes if the precondition is satisfied
Design by contract: contract between function and code calling the function - if code passes parameters satisfying preconditions, function will produce postcondition
Specification-based testing: tests designed based on only information in the function specification, not considering function implementation
Test-first development: writing unit test before writing the function

- ★ Ensures unit tests are written
- ★ Less bugs in code
- ★ Helps clarify function specification
- ★ Helps quickly fix problems
- ★ Quicker development time

Pytest: unit testing framework in python

- ★ Use **test**_ prefix on unit test function

Iteration

Iteration: repeated execution of a sequence of statements
While loop: uses boolean statement, body repeats while expression evaluates to True

- ★ Use while loop instead of for loop if you want program to repeat indefinitely until a certain condition is met
- ★ Use to validate input to a function

Sentinel value: value that signals the end of the loop
Image processing using iteration

- ★ Images made up of grid of pixels, color by RGB color model - three values 0-255 representing intensity of red, green, blue color
- ★ Use nested iteration: inner and outer loop

Strings

String: sequential collection of characters

- ★ Strings are immutable, can't change existing string, only create new string which is variation on original

Empty string: string that contains no characters ""
+ operator: concatenation, join strings together
* operator: repetition
Index operator: use to select specific characters from string

- ★ string_name[1]
- ★ Starts at 0 - first letter is index 0
- ★ Negative index: use negative number in index to count backward from end of string

String Methods:

Upper: create new string with all uppercase characters
Lower: create new string with all lowercase characters
Capitalize: first character capitalized
Strip: remove first and last whitespace
Lstrip/Rstrip: remove leading or trailing whitespace
Count: get number of occurrences of character(s)
Replace: replace old substring with new substring
Center: get string in provided width of center characters
Ljust/Rjust: string left/right justified in field of certain width
Find/Rfind: find leftmost or rightmost index where item is found
Index/Rindex: find leftmost or rightmost index where item is found, causes runtime error if not found
Format: method to substitute string into another string in place of {}
Len: find number of characters in a string
Slice: substring within a string

- ★ Use index to select slice: string[n:m] part of string from nth to mth character

Each character assigned ordinal value, **ord** function finds ordinal value for given character
Chr: convert integer to character equivalent
Traversal: use **for** loop to iterate through characters in a string

- ★ Can also use **while** loop

in operator: test if one string is substring of another

Lists

List: sequential collection of data values called elements, denoted by [ ]

- ★ Lists can be nested within another list, inner list is sublist
- ★ **In** and **not in**: boolean operators that tell whether element is in list
- ★ + concatenates lists
- ★ * repeats items in list given number of times
- ★ Lists are mutable
- ★ Can be parameters for functions

Tuple: sequence of items that is immutable

List Methods:

len: return length of list
id: return identification tag of object
del: remove element from list
append: adds something to end of list
insert: insert item into certain position
pop: remove and return last item in list
sort: modifies list to be sorted
reverse: puts list in reverse order
index: return index of first occurrence of item
count: return number of occurrences of items
remove: remove first instance of item
split: break string into list of characters
list: convert into list

Dictionaries

Dictionary: mapping type collection with pairs of **keys** and **values**

- ★ Denoted by {}

Dictionary Methods:

del: delete key-value pair from dictionary
keys: returns all keys in dictionary
values: returns values in dictionary
items: returns key-value pairs in dictionary
get: returns value/key associated with key/value

Files

\n: new line

File Methods:

open(filename, ‘r’): open file for reading

open(filename, ‘w’): open file for writing
filevariable.close(): close file
readline: read one line, return it as a string
write: add string to end of file that has been opened for writing
read(n): read and return string of n characters, whole file if no n

Classes and Objects

Object-oriented programming: uses objects with data and functionality

- ★ Each object has a state and collection of methods it can perform

Initializer method **__init__**: assign initial state values to a new instance
Constructor: function that creates new object instance
Instantiation: process of making new objects and initializing default settings
String method __str__: define string to be returned as representation of object

Fractions

Deep copy: copy contents of object and embedded objects
Shallow copy: copy contents of object and references to embedded objects
Deep equality: references that point to objects with the same value
Shallow equality: references that point to the same object

Recursion

Recursion: method of solving problems that breaks them into smallest possible problem, uses a function that calls itself

- ★ Alternative to using **while** and **for** loops
- ★ Must have a base case, change state to move toward base case
- ★ Can be visualized using fractals